# VESPER — ERS actualizado (versión 2)

**Asignatura:** DSY1104, Desarrollo Fullstack II
**Entrega:** Evaluación Parcial 2, frontend y pruebas unitarias
**Repositorio:** https://github.com/Go-tei/vesper\ **Integrantes:** Enzo Carggiolis, Fernando Campos y Marcelo Perez.
**Sección:** DSY1104.
**Fecha de actualización:** 5 de noviembre de 2026.
**Fecha de presentación:** 5 de noviembre de 2026.

## 1. Propósito y fuentes

VESPER es una tienda académica de streetwear inspirada en series, cine, música, anime y videojuegos. Esta versión transforma el sitio de la primera entrega, basado en HTML y JavaScript, en una aplicación React. Conserva la identidad visual y el catálogo existente, y amplía la navegación, la administración y el proceso de compra.

Esta especificación traduce los requisitos de los documentos proporcionados: **“Evaluación Parcial N.º 2: Implementando frontend y pruebas unitarias a mi tienda online”** y **“Instrucciones para el desarrollo de la Evaluación 2”**. La pauta solicita React, Bootstrap, persistencia simulada en JavaScript, pruebas con Jasmine y Karma, ERS actualizado, informe de testing, repositorio público y proyecto comprimido.

El documento define comportamiento y criterios de aceptación. Los resultados efectivos de ejecución y el alcance medido de las pruebas deben consultarse en el informe de testing de esta entrega; una lista de requisitos no constituye evidencia de ejecución.

## 2. Alcance y supuestos

El alcance es una aplicación frontend ejecutada en el navegador. Utiliza un archivo JavaScript de datos iniciales y un servicio de persistencia local. Permite explorar productos sin iniciar sesión, gestionar un carrito, simular una compra y administrar los datos de la tienda mediante una cuenta de demostración.

Para esta entrega se adoptan estos supuestos:

- Se dispone del repositorio de VESPER de la primera entrega y de los dos PDF. No se proporcionó un backend, una API ni el template administrativo HTML adicional mencionado en las instrucciones.
- El archivo JavaScript de datos simulados solicitado por la pauta se implementa con persistencia en `localStorage`; `sessionStorage` conserva la sesión de la pestaña. Estos datos pertenecen al navegador y al origen de la aplicación.
- El pago se simula de forma explícita y permite demostrar tanto aprobación como rechazo. No se solicitan tarjetas ni se procesan pagos reales.
- El despacho tiene una tarifa de demostración de $3.990 CLP; el retiro tiene costo cero. Los precios se expresan en pesos chilenos.
- Los reportes se calculan con los pedidos registrados localmente. No representan ventas de un sistema externo.

- La autenticación y la restricción visual de rutas son educativas. Las cuentas y contraseñas de demostración se almacenan en el cliente; esta solución no proporciona seguridad de producción.

La mención a una presentación con backend y frontend se cubre aquí explicando la separación entre interfaz y servicio simulado. Una API real, autenticación de servidor, base de datos remota y pasarela de pago requieren una etapa adicional y no se declaran implementadas.

## 3. Actores

| Actor | Facultades |
|---|---|
| Visitante | Consulta la tienda pública, busca productos, utiliza el carrito, realiza un checkout simulado, envía una consulta local y puede registrarse. |
| Cliente registrado | Accede a las funciones públicas, inicia/cierra sesión, mantiene su perfil y utiliza sus datos para completar el checkout. |
| Administrador | Accede al panel, mantiene productos, categorías y usuarios, consulta pedidos, resúmenes, stock crítico y su perfil. |

## 4. Requisitos funcionales y aceptación

Los identificadores permiten relacionar funcionalidad, código, demostración y casos del informe de testing.

| ID | Requisito | Criterio de aceptación |
|---|---|---|
| RF-01 | Catálogo público en React | Un visitante puede abrir la tienda y ver los productos con imagen, nombre, categoría y precio sin iniciar sesión. Las listas se generan a partir de datos, mediante componentes reutilizables. |
| RF-02 | Búsqueda y filtros | Escribir una consulta o elegir una categoría actualiza los productos visibles. Si no hay coincidencias se presenta un estado vacío comprensible. |
| RF-03 | Detalle de producto | Desde un producto se accede a su información individual, disponibilidad y acción de compra; un identificador inexistente muestra una salida de recuperación. |
| RF-04 | Categorías y ofertas | Hay una vista de categorías con acceso a sus productos y una vista de ofertas que presenta productos con descuento y su precio correspondiente. |
| RF-05 | Contenido y navegación | La aplicación ofrece inicio, catálogo, nosotros, blog y detalle de artículo, contacto, acceso y registro; los enlaces internos cambian de vista sin recargar el documento. |
| RF-06 | Registro y acceso | Se valida nombre, correo y contraseña. No se admite un correo duplicado. Credenciales incorrectas muestran un error; las correctas crean una sesión sin exponer la contraseña en la navegación. |
| RF-07 | Perfil y cierre de sesión | Una cuenta conectada puede actualizar los datos permitidos de su perfil. Cerrar sesión actualiza el menú y restringe el acceso administrativo. |
| RF-08 | Carrito persistente | Agregar productos, modificar cantidades o quitar líneas actualiza subtotales, total y contador. Las cantidades deben ser enteras y respetar el stock y el límite de 20 unidades por producto. Recargar conserva el carrito local válido. |
| RF-09 | Checkout validado | El checkout presenta el resumen e identifica al cliente. Para despacho solicita datos de dirección y permite seleccionar la entrega. Un carrito vacío no puede generar un pedido. |
| RF-10 | Autocompletado | Si hay una sesión con datos personales y dirección, el checkout se inicia con esos valores, que pueden revisarse antes de confirmar. |

| ID | Requisito | Criterio de aceptación |
|---|---|---|
| RF-11 | Compra aprobada | Una aprobación simulada crea un pedido con fecha, cliente, detalle y totales; descuenta stock y vacía el carrito. La vista de éxito permite revisar el resumen. |
| RF-12 | Compra rechazada | Un rechazo simulado lleva a la vista de error y permite reintentar. No crea un pedido, no descuenta stock y conserva el carrito. |
| RF-13 | Administración de productos | Un administrador puede listar, crear, editar y eliminar productos. Se validan los campos requeridos, la categoría, el precio positivo, stock entero no negativo y descuento permitido. Los cambios se reflejan en la tienda y persisten al recargar. |
| RF-14 | Administración de categorías | Un administrador puede crear, consultar, editar y eliminar categorías respetando las relaciones con productos y evitando nombres inválidos o duplicados. |
| RF-15 | Administración de usuarios | Un administrador puede consultar y mantener cuentas y roles. Se rechazan correos duplicados y acciones que dejen al sistema sin administrador o eliminen/degraden la cuenta administrativa activa. |
| RF-16 | Pedidos y resúmenes | El panel lista los pedidos aprobados y permite consultar un detalle tipo boleta, conservando el nombre y precio de cada producto al momento de la compra. No se presenta como documento tributario. |
| RF-17 | Historial y reportes | Se puede consultar el historial de compras de una cuenta y reportes derivados de los pedidos. Se identifican productos con stock crítico, definido como cinco unidades o menos. |
| RF-18 | Acceso administrativo | Sin sesión se solicita iniciar sesión para acceder al panel; un cliente sin rol administrador no accede a las funciones administrativas de la interfaz ni a sus operaciones del servicio. |
| RF-19 | Contacto | El formulario valida datos y registra una consulta local. El resultado informa su carácter local; no afirma haber enviado un correo. |
| RF-20 | Persistencia e integridad | Los datos iniciales se cargan cuando faltan y se conservan las modificaciones válidas anteriores. El JSON local ilegible se recupera mediante datos de respaldo; también se normalizan productos, usuarios y líneas de carrito al iniciar. El estado publicado por el servicio actualiza los componentes suscritos. |

# 5. Mapa de navegación

Las rutas se declaran en `src/App.jsx`. El punto de entrada `src/main.jsx` utiliza `HashRouter`: las direcciones se escriben después de `/#`, por ejemplo `/#/productos`. Así la navegación funciona en alojamiento estático sin exigir que el servidor resuelva cada ruta de React.

| Grupo | Rutas |
|---|---|
| Tienda | `/`, `/productos`, `/productos/:id`, `/categorias`, `/categorias/:nombre`, `/ofertas` |
| Contenido | `/nosotros`, `/blog`, `/blog/:slug`, `/contacto` |
| Cuenta | `/login`, `/registro`, `/perfil` |
| Compra | `/carrito`, `/checkout`, `/compra/:id`, `/pago-error` |
| Panel | `/admin`, `/admin/productos`, `/admin/productos/nuevo`, `/admin/productos/:id` |
| Mantenedores | `/admin/categorias`, `/admin/usuarios`, `/admin/usuarios/nuevo`, `/admin/usuarios/:id` |
| Operación | `/admin/ordenes`, `/admin/ordenes/:id`, `/admin/reportes`, `/admin/perfil` |

Flujo de compra: catálogo o categoría → detalle o carrito → checkout → compra aprobada o pago rechazado → resumen o reintento.

Flujo administrativo: acceso con cuenta administradora → panel → mantenedores, órdenes o reportes → consulta o formulario → actualización de datos locales y de la interfaz.

## 6. Requisitos no funcionales

| ID | Requisito | Criterio verificable |
|---|---|---|
| RNF-01 | Tecnología y estructura | React gestiona las vistas; Vite procesa JSX y genera el build; Bootstrap aporta estilos y rejilla. Datos, servicio, contexto, componentes y páginas tienen responsabilidades separadas. |
| RNF-02 | Diseño responsivo | En móvil, tableta y escritorio, los elementos se reorganizan mediante la rejilla y utilidades de Bootstrap. Los formularios mantienen etiquetas legibles, el menú es utilizable y las tablas pueden desplazarse dentro de su contenedor. |
| RNF-03 | Accesibilidad básica | Los campos tienen etiquetas, las imágenes de productos tienen texto alternativo, los controles usan elementos semánticos y los estados relevantes se anuncian con mensajes comprensibles. |
| RNF-04 | Mantenibilidad | Los componentes reciben datos y callbacks por props; el estado de formularios y filtros permanece cerca de su uso. El servicio centraliza validación y persistencia para evitar reglas duplicadas en las páginas. |
| RNF-05 | Pruebas ejecutables | Jasmine define los casos y aserciones; Karma los ejecuta en un navegador. La suite cubre renderizado, props, estado, eventos, condiciones y reglas relevantes del servicio. La pauta exige demostrar al menos 10 pruebas correctas. |
| RNF-06 | Reproducibilidad | El repositorio incluye dependencias y scripts, instrucciones de instalación y ejecución, documentos y archivo comprimido de código fuente. Se registran los comandos y resultados reales de verificación. |
| RNF-07 | Transparencia de la simulación | La interfaz y los documentos distinguen datos locales, consultas locales y pagos simulados de un servicio remoto. |

## 7. Diseño de la solución

La migración reemplaza páginas HTML independientes por una aplicación de componentes. La interfaz conserva el lenguaje visual de VESPER y reutiliza sus imágenes. React Router representa cada vista mediante una ruta; el panel agrupa sus rutas de administración.

La organización separa estas responsabilidades:

- `src/data/seed.js`: datos JavaScript iniciales de productos, usuarios y categorías.
- `src/services/store.js`: CRUD, validaciones, sesión, carrito, checkout y persistencia. Expone un snapshot y una suscripción a cambios.
- `src/context/ShopContext.jsx`: publica el estado compartido a través de React. `useSyncExternalStore` conecta los componentes con el snapshot estable del servicio; al recibir una notificación React vuelve a leerlo.
- `src/components/ProductCard.jsx` y `ProductList.jsx`: reciben `product` o `products` y el callback `onAdd`; presentan tarjetas y listas con estados de oferta, disponibilidad y lista vacía.
- `src/components/Field.jsx` y `Feedback.jsx`: reutilizan etiquetas, campos controlados y mensajes condicionales. `Layout.jsx` comparte navegación, pie y título de página.
- `src/pages/CatalogPage.jsx`: combina productos y categorías con búsqueda y filtro en los parámetros de URL `buscar` y `categoria`; recibe `showHero` y `offersOnly` para reutilizarse en inicio y ofertas.

- `src/pages/ProductPage.jsx`: consulta el identificador de la ruta y mantiene localmente cantidad y mensajes.
- `src/pages/AuthPages.jsx`: contiene `AuthPage` y `ProfilePage`, con formularios de acceso, registro y perfil; el perfil muestra el historial propio.
- `src/pages/CartPages.jsx`: contiene `CartPage`, `CheckoutPage`, `Receipt`, `SuccessPage` y `PaymentErrorPage`; separa la edición del carrito, datos de entrega y resultado simulado.
- `src/pages/ContactPage.jsx`, `InfoPages.jsx` y `src/data/articles.js`: contacto, categorías, nosotros, blog, artículos y página no encontrada.
- `src/pages/AdminPages.jsx`: agrupa rutas y componentes de resumen, formularios de producto y usuario, categorías, pedidos, comprobantes, reportes y perfil administrativo.

Esta separación permite cambiar una regla, como el límite de cantidad, en el servicio y comprobarla independientemente de la apariencia. Los formularios controlados representan cada valor en estado React; las validaciones devuelven errores visibles en la interfaz. Las operaciones generan un estado candidato y lo persisten antes de publicarlo; un error de almacenamiento evita publicar una compra incompleta. Los datos compartidos se guardan juntos bajo `vesper_store_v2` y la sesión de la pestaña bajo `vesper_session`.

## 7.1 Modelo de datos

| Entidad | Datos principales y relaciones |
|---|---|
| Producto | `id`, `nombre`, `categoria`, `precio`, `descripcion`, `imagen`, `stock`, `descuento`. La categoría debe corresponder a una categoría existente. |
| Categoría | `id`, `nombre`. Agrupa productos para navegación y administración. |
| Usuario | `id`, `nombre`, `email`, `password` de demostración, `role`, `telefono`, `direccion`, `comuna`, `region`. |
| Sesión | Identidad y datos permitidos de la cuenta activa, sin contraseña. Se mantiene por pestaña. |
| Línea de carrito | `productId`, `cantidad`. Obtiene precio y disponibilidad del producto vigente. |
| Pedido | `id`, `fecha`, `userId` opcional, `cliente`, `items`, `subtotal`, `envio`, `total`. Cada ítem conserva nombre, precio y cantidad al confirmar. |
| Consulta | Datos validados del formulario de contacto, registrados localmente. |

El total se calcula con el precio unitario aplicable, incluyendo el descuento del producto, multiplicado por la cantidad, más el costo de entrega. La compra se valida nuevamente al confirmar para evitar confirmar productos inexistentes o cantidades superiores al stock.

## 7.2 Restricciones de almacenamiento

La persistencia no se comparte entre computadoras o navegadores. Limpiar los datos del sitio elimina la información local. `localStorage` no reemplaza una base de datos segura; cualquier usuario con herramientas del navegador puede inspeccionarlo o modificarlo. No deben utilizarse datos personales ni contraseñas reales durante la demostración.

## 8. Relación con la rúbrica

| Indicador | Evidencia a presentar |
|---|---|
| IE2.1.1 e IE2.1.3 | Estructura React y navegación funcional; explicación del motivo de la migración y separación de responsabilidades. |
| IE2.1.2 e IE2.1.4 | Un componente reutilizable con props, un formulario con estado controlado y la misma vista a diferentes anchos con Bootstrap. |
| IE2.2.1 e IE2.2.2 | Suite Jasmine/Karma, al menos 10 pruebas correctas y ejemplos de renderizado, estado y eventos. |
| IE2.3.1 e IE2.3.2 | Configuración del entorno, aislamiento mediante datos de prueba o mocks, ejecución y análisis del informe de testing. |

La configuración está en `karma.conf.cjs`; `tests/index.js` carga las especificaciones y los módulos de `src` para que las vistas sin ejecutar también aparezcan en el denominador de cobertura. `src/main.jsx`, que monta la aplicación en el navegador, queda fuera de esa instrumentación. Los casos se agrupan en `tests/components.spec.jsx`, `tests/admin.spec.jsx` y `tests/store.spec.js`, con utilidades DOM en `tests/helpers.jsx`.

La cobertura se debe expresar con precisión: cantidad de casos ejecutados, funcionalidades y escenarios comprobados, y, si se genera instrumentación, porcentajes medidos sobre los archivos incluidos. Ninguno de esos valores equivale por sí solo a ausencia total de defectos.

## 9. Entregables y revisión de cierre

1. Repositorio público actualizado: `https://github.com/Go-tei/vesper`.
2. Proyecto frontend comprimido con fuentes, archivos de configuración e instrucciones; excluir dependencias instaladas y datos privados.
3. Esta ERS V2 actualizada, con integrantes, sección y fecha de presentación completos.
4. Informe de cobertura de testing con casos, resultados, alcance y limitaciones reales.
5. Presentación de 15 minutos y preparación para 5 minutos de preguntas individuales.

Antes de presentar, verificar el catálogo público, ambos resultados del checkout, los mantenedores con cuenta administradora, la persistencia después de recargar y el funcionamiento en diferentes tamaños de pantalla. Ejecutar nuevamente los comandos de prueba y build documentados en el repositorio y conservar sus resultados.